
BỘ TÀI LIỆU KỸ THUẬT · TÀI LIỆU 03

LAAM

Hạ tầng, Bảo mật & Vận hành

Trợ lý AI hội thoại cho công việc được xây dựng, triển khai, bảo mật và vận hành như thế nào ở thời điểm hiện tại, và những gì đã được lên kế hoạch trước khi chạy production.



PHIÊN BẢN
1.0

NGÀY
2026-09-24

PHIÊN BẢN MÃ NGUỒN
LAAM@7eb0724

Mục lục

1 Cấu trúc triển khai

Stack tham chiếu · dịch vụ chạy trực tiếp trên host · ingress · tick bộ lập lịch bên ngoài

2 Runtime & build

Image Next.js standalone · Node 24 · migration Drizzle · health check

3 Cấu hình

Nhóm biến môi trường · quản lý secret

4 Mô hình mở rộng & ràng buộc hiện tại

Giả định một tiến trình · cấu trúc production mục tiêu (đề xuất)

5 Kiến trúc bảo mật

Xác thực · phân quyền · cô lập dữ liệu · mã hoá thông tin xác thực · OAuth cho connector · SSRF · cổng xác nhận ghi · cấm thực thi mã · header · MCP server · bảng kiểm soát

6 Bảo vệ dữ liệu & luồng dữ liệu AI

Dữ liệu nào rời khỏi hạ tầng nội bộ, đến đâu và khi nào · cấu hình hoàn toàn nội bộ

7 Vận hành

Khả năng quan sát · theo dõi chi phí · sao lưu · cập nhật & migration · health check

8 Hạng mục gia cố trước khi lên production

Thuật ngữ trạng thái dùng trong tài liệu

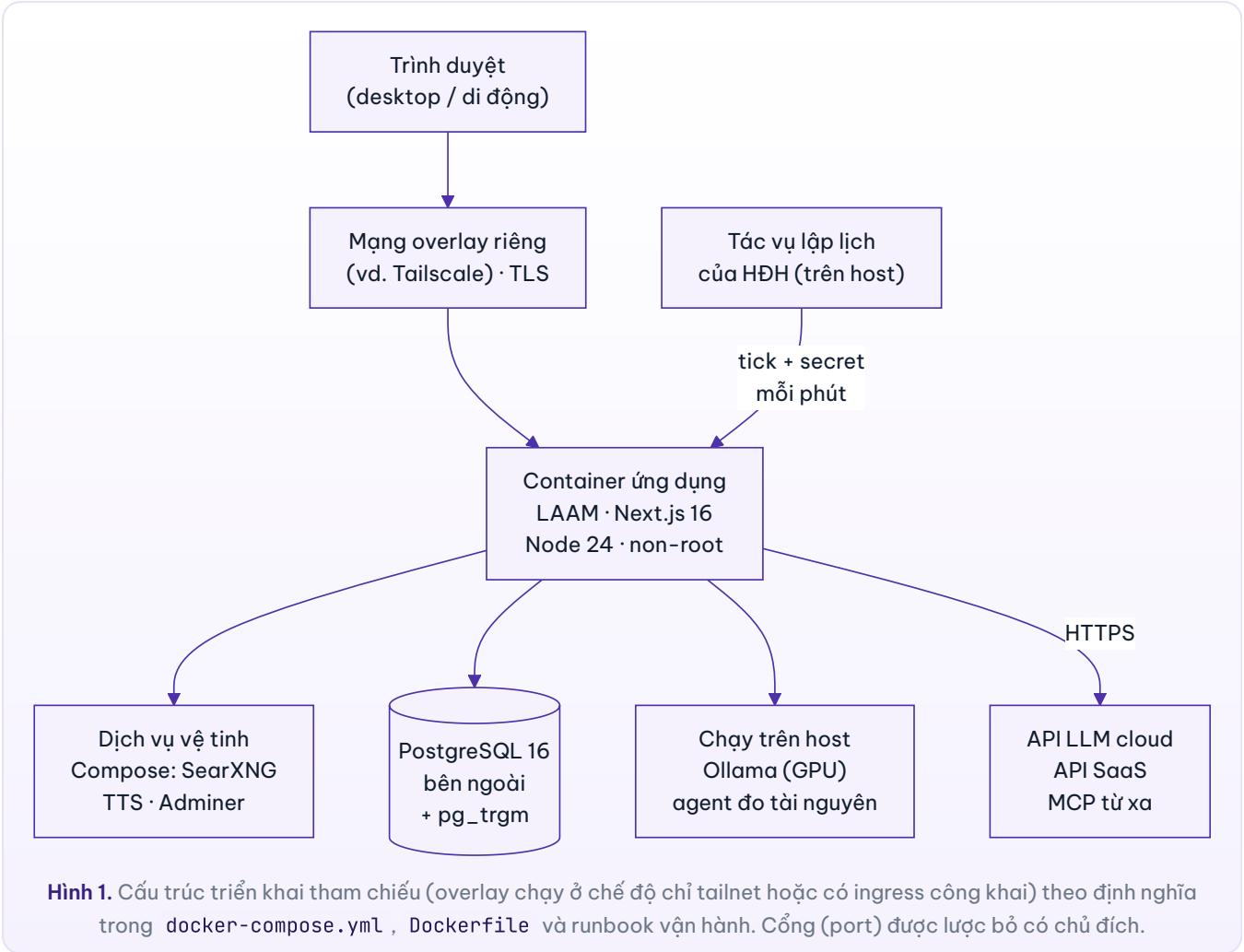
LIVE đã hiện thực, đã tích hợp và đã chạy với dữ liệu thật trên stack tự host · **BUILT** đã hiện thực và đã tích hợp, chưa chạy với dữ liệu production · **PLANNED** nằm trong lộ trình, đang tắt bằng cờ tính năng hoặc chưa hiện thực.

Trạng thái triển khai: giai đoạn tiền production (chưa triển khai production)

Hiện nay LAAM chạy dưới dạng stack Docker Compose tự host, phục vụ phát triển, sử dụng nội bộ và trình diễn. Chưa có môi trường production (repository gọi bản triển khai tự host nội bộ là stack "production"; bản này không phục vụ khách hàng nào), và tài liệu này không đưa ra tuyên bố nào về khách hàng production, SLA, uptime hay quy mô đã đo lường. Mục 4.3 mô tả một cấu trúc production **đề xuất**, và Mục 8 liệt kê các công việc gia cố đã lên kế hoạch trước khi ra mắt production.

1 Cấu trúc triển khai

LAAM là một ứng dụng Next.js 16 duy nhất, dùng PostgreSQL 16 làm nền dữ liệu. Xung quanh là ba container nhỏ (tìm kiếm web, chuyển văn bản thành giọng nói và giao diện quản trị cơ sở dữ liệu), hai dịch vụ chạy trực tiếp trên host vì cần truy cập GPU và host, cùng một tick lập lịch từ bên ngoài.



1.1 Thành phần

Thành phần	Chạy dưới dạng	Vai trò	Trạng thái
Ứng dụng LAAM	Service Compose, container non-root	Toàn bộ trang giao diện và 64 file API route. Chứa vòng lặp agent cho chat và giọng nói, workflow engine, các connector, MCP client và MCP server. Container công bố một cổng HTTP ra host.	LIVE
PostgreSQL 16	Instance dùng chung bên ngoài, truy cập qua mạng Docker	Hệ thống lưu trữ gốc: 22 bảng, tìm kiếm trigram bằng pg_trgm . LAAM không còn chạy container cơ sở dữ liệu riêng; dùng stack không ảnh hưởng đến cơ sở dữ liệu và dữ liệu bên trong.	LIVE
SearXNG	Service Compose, chỉ lắng nghe trên loopback	Công cụ metasearch tự host phía sau tool web_search . Ứng dụng không phụ thuộc vào nó khi khởi động, nên tìm kiếm web suy giảm êm khi dịch vụ này ngừng.	LIVE

Thành phần	Chạy dưới dạng	Vai trò	Trạng thái
VieNeu TTS	Service Compose (Python/FastAPI), chỉ lắng nghe trên loopback, giới hạn 3 CPU	Chuyển văn bản thành giọng nói bằng mạng nơ-ron, dạng streaming, cho giao diện giọng nói (giọng tiếng Việt và tiếng Anh). Giới hạn CPU và giới hạn luồng được đặt đồng thời. Đo trên host tham chiếu, dịch vụ chạy nhanh gấp 3.58× thời gian thực mà không làm trình duyệt thiếu tài nguyên.	LIVE
Adminer	Service Compose	Giao diện web để người vận hành quản trị cơ sở dữ liệu. Cổng của nó được công bố trên host (xem Mục 8, hạng mục 7).	LIVE
Ollama	Chạy trực tiếp trên host (cần GPU)	Runtime LLM cục bộ: lựa chọn mô hình tại chỗ, chi phí \$0. Container truy cập nó qua Docker host gateway.	LIVE
Agent đo tài nguyên host	Chạy trên host, script Node không phụ thuộc thư viện	Lấy mẫu CPU, RAM và GPU (qua <code>nvidia-smi</code>) khoảng mỗi giây một lần cho bảng hệ thống của trung tâm điều khiển giọng nói. Được đọc qua một proxy trong ứng dụng, có cơ chế suy giảm êm; hỗ trợ bearer token tùy chọn.	BUILT
Tick workflow	Tác vụ lập lịch của HĐH	Gửi POST đến endpoint tick mỗi phút kèm một secret dùng chung. Tick này nhận các lịch đến hạn và tiếp tục các lượt chạy bị gián đoạn. Không có cron trong tiến trình.	LIVE
Collector giám sát	Script Node không phụ thuộc thư viện trên từng máy trạm được giám sát	Đẩy metadata phiên làm việc của coding agent đã được phân tích cùng các đoạn văn bản ngắn (không bao giờ gửi toàn bộ transcript) lên endpoint ingest, dùng collector token riêng của từng máy. Tự thử lại với backoff có jitter.	BUILT

1.2 Ingress và mức độ phơi bày mạng

- Ingress** được thiết kế để đi qua một mạng overlay riêng (ví dụ Tailscale). Overlay kết thúc TLS và có hai chế độ: *chỉ tailnet*, chỉ các thiết bị đã đăng ký truy cập được, dùng cho phát triển; và *ingress công khai* cho trình diễn. Ứng dụng tin tưởng host được chuyển tiếp (`trustHost`), đúng yêu cầu của một triển khai tự host phía sau proxy. Cổng của ứng dụng và của giao diện quản trị cơ sở dữ liệu cũng được công bố trên host, nên mức phơi bày trong mạng cục bộ do tường lửa của host quyết định (Mục 8, hạng mục 7).
- Phụ thuộc nội bộ** (SearXNG, TTS) chỉ được công bố trên giao diện loopback. Ứng dụng truy cập chúng qua tên service Compose.
- Lưu lượng ra ngoài** từ ứng dụng đi đến các nhà cung cấp LLM cloud, API SaaS của connector, MCP server từ xa do người dùng cấu hình và các dịch vụ địa lý/thời tiết công khai không cần khoá. Tất cả đều phụ thuộc vào cấu hình; xem Mục 6.

Chế độ phát triển

Lập trình viên chạy cùng mã nguồn bằng `npm run dev` (Turbopack) trên cùng schema cơ sở dữ liệu. Docker image được dùng để host bản dùng nội bộ và bản trình diễn.

2 Runtime & build

Node 24

RUNTIME ALPINE
(ACTIVE LTS)

3 giai đoạn

DOCKER BUILD · DEPS →
BUILD → RUN

19

MIGRATION SQL DRIZZLE
(CÓ JOURNAL)

non-root

NGƯỜI DÙNG TRONG
CONTAINER

2.1 Runtime ứng dụng

Tầng	Lựa chọn
Framework	Next.js 16.2 App Router, <code>output: "standalone"</code> . Image mang theo một <code>server.js</code> tự chứa, không cần toàn bộ <code>node_modules</code> hay <code>next start</code> .
Giao diện · kiểm tra dữ liệu	React 19.2, TypeScript 6 (strict), Tailwind CSS 4; <code>zod</code> 4 tại ranh giới request
Xác thực	Auth.js v5 beta (<code>next-auth</code> 5.0.0-beta.31) với Drizzle adapter; <code>bcryptjs</code> để băm mật khẩu
Truy cập dữ liệu	<code>drizzle-orm</code> 0.45 trên <code>node-postgres</code> (<code>pg</code> , được giữ là package <code>server-external</code> nên không bị bundle)
AI / MCP	Anthropic SDK cho Claude; các nhà cung cấp tương thích OpenAI qua <code>fetch</code> thuần (không dùng SDK); <code>@modelcontextprotocol/sdk</code> cho MCP client (Streamable HTTP, dự phòng SSE); MCP server của chính LAAM là một bộ xử lý JSON-RPC viết tay trên Streamable HTTP
Binary hệ thống trong image	<code>poppler-utils</code> (trích xuất văn bản và raster hoá PDF phía server), <code>tesseract-ocr</code> với dữ liệu tiếng Anh, tiếng Việt và tiếng Trung giản thể (OCR), <code>unzip</code> (trích xuất DOCX)

2.2 Build container

- deps:** `npm ci` từ lockfile trên `node:24-alpine`. Phiên bản major của Node/npm được cố định trùng với máy phát triển, tránh lệch định dạng lockfile.
- builder:** `next build` với `NODE_ENV=production` và tắt telemetry. Tài nguyên tĩnh được sao chép vào đầu ra `standalone`.
- runner:** image tối giản. Các package OCR và PDF được cài bằng `root`, sau đó image chuyển sang người dùng không đặc quyền có sẵn `node` trước khi sao chép ứng dụng vào. Điểm khởi chạy là `node server.js`.

Image do người vận hành build (`docker build`) và gắn tag cục bộ. Hiện chưa có container registry hay pipeline CI/CD (Mục 8).

2.3 Migration cơ sở dữ liệu

- Schema nằm trong một file TypeScript (`src/db/schema.ts`). Migration SQL được sinh bằng `drizzle-kit generate`, được commit, và áp dụng bằng `drizzle-kit migrate`. Journal là nguồn chuẩn xác định migration nào đã chạy.
- Chính sách:** dữ liệu thật chỉ được thay đổi qua migration. Không bao giờ dùng `db.push` trên cơ sở dữ liệu thật. Hai migration được viết tay và có tính idempotent: index hiệu năng, và extension `pg_trgm` kèm index trigram GIN. Tìm kiếm trigram được chọn thay cho `tsvector` vì PostgreSQL không có sẵn cấu hình full-text cho tiếng Việt hay tiếng Trung.

- **Thực thi:** người vận hành chạy migration từ host trước khi thay container ứng dụng. Runtime image không chạy migration khi khởi động và không chứa `drizzle-kit`.
- **Hook khởi động:** `instrumentation.register()` chạy một lần cho mỗi tiến trình và đánh dấu các lượt chạy workflow còn ở trạng thái `running` do sự cố là `resumable`; tick kế tiếp tiếp tục chúng từ journal các bước. Lỗi cơ sở dữ liệu không thể chặn việc khởi động.

3 Cấu hình

Cấu hình chỉ dùng biến môi trường, theo phong cách twelve-factor. `.env.example` mô tả các biến dành cho người vận hành. File Compose nạp `.env` phía host và ghi đè các URL dịch vụ khác biệt bên trong mạng container. Các tích hợp được bật/tắt theo sự hiện diện của thông tin xác thực: nếu không đặt khoá của một nhà cung cấp, nhà cung cấp đó đơn giản là bị ẩn.

Nhóm	Biến tiêu biểu (chỉ tên)	Tác dụng
Cơ sở dữ liệu	<code>DATABASE_URL</code>	Kết nối PostgreSQL
Xác thực	<code>AUTH_SECRET</code> , <code>AUTH_URL</code> , <code>AUTH_TRUST_HOST</code>	Secret ký JWT và URL gốc công khai
Đăng ký	<code>REGISTER_MODE</code> , <code>REGISTER_INVITE_CODE</code>	Đăng ký ở chế độ <i>open</i> , <i>invite</i> hoặc <i>closed</i>
Mô hình cục bộ	<code>OLLAMA_URL</code> , <code>DEFAULT_CHAT_MODEL</code> , <code>CHAT_NUM_CTX</code>	Endpoint LLM cục bộ và mô hình mặc định
Nhà cung cấp LLM cloud	<code>BYTEPLUS_*</code> , <code>CEREBRAS_*</code> , <code>DEEPSEEK_*</code> , <code>ANTHROPIC_API_KEY</code>	Có khoá thì nhà cung cấp đó được bật; base URL và độ trễ thử lại có thể tinh chỉnh
Bộ định tuyến mô hình nội bộ	<code>INTERNAL_MODEL</code> , <code>INTERNAL_JSON_MODEL</code>	Cố định mô hình dùng cho tác vụ AI chạy nền (ghi đè thứ tự ưu tiên cloud)
Tinh chỉnh vòng lặp agent	<code>CHAT_MAX_ROUNDS</code> , <code>CHAT_REASONING_EFFORT_*</code> , <code>DIGEST_*</code> , <code>TOOL_*</code>	Chặn vòng lặp vô hạn, mức độ suy luận, ngân sách ngữ cảnh, các guard định tuyến tùy chọn bật
Secret & OAuth của connector	<code>CONNECTOR_KEY</code> ; <code>GOOGLE_OAUTH_*</code> , <code>ATLASSIAN_OAUTH_*</code> , <code>SLACK_*</code> , <code>ZALO_*</code> , <code>TRELLO_API_KEY</code> , <code>OAUTH_PUBLIC_BASE_URL</code>	Khoá chủ để mã hoá thông tin xác thực (dùng secret xác thực thay thế khi không đặt; xem Mục 5.4); mỗi connector OAuth chỉ xuất hiện khi client của nó đã được cấu hình
Bộ lập lịch & an toàn workflow	<code>WORKFLOW_TICK_SECRET</code> , <code>WORKFLOW_*_ALLOWLIST</code>	Xác thực tick; whitelist người nhận theo từng kênh cho tin nhắn gửi ra theo lịch
Dịch vụ vệ tinh	<code>SEARXNG_URL</code> , <code>CONSTELLATION_TTS_URL</code> , <code>HOST_METRICS_URL</code> , <code>HOST_METRICS_TOKEN</code>	Endpoint tìm kiếm web, TTS và đo tải nguyên host
Giám sát & cảnh báo	<code>LAAM_STUCK_MIN</code> , <code>LAAM_PROJECTS_DIR</code> , <code>PROACTIVE_*</code>	Ngưỡng cảnh báo phiên bị treo và chi phí; vị trí log phiên cục bộ
Collector (phía client)	<code>LAAM_URL</code> , <code>LAAM_MACHINE_TOKEN</code> , <code>LAAM_INTERVAL_SEC</code>	Đặt trên từng máy trạm được giám sát, không đặt trên server

3.1 Quản lý secret hiện nay

- Secret của server (`AUTH_SECRET`, `CONNECTOR_KEY`, khoá API của nhà cung cấp, client secret OAuth và `WORKFLOW_TICK_SECRET`) nằm trong file `.env` của host. File này được loại khỏi hệ thống quản lý phiên

bản. Chuỗi kết nối cơ sở dữ liệu trong mạng container hiện được đặt trong file Compose (Mục 8, hạng mục 7). **Không secret nào được gửi xuống trình duyệt.** Lời gọi LLM, lời gọi connector và trao đổi token OAuth đều diễn ra phía server.

- Secret theo từng người dùng (token connector và thông tin xác thực của MCP server từ xa) **không** phải biến môi trường. Chúng được lưu mã hoá trong PostgreSQL (Mục 5.4) và được che trong mọi phản hồi API, chỉ hiển thị bốn ký tự cuối.
- Phần lớn biến được đọc khi tiến trình khởi động, nên thay đổi whitelist hay secret cần khởi động lại container ứng dụng. Runbook có ghi rõ điều này.

4 Mô hình mở rộng & ràng buộc hiện tại

Bản build hiện tại được thiết kế cho **đúng một tiến trình ứng dụng**, và mã nguồn nêu rõ điều này ở nhiều chỗ. Trên một host, cách làm này đơn giản và đúng đắn. Đây cũng là thay đổi chính cần thực hiện trước khi lên production: một số trạng thái nằm trong bộ nhớ tiến trình, và một thủ tục khởi động giả định nó là bộ thực thi duy nhất.

4.1 Các giả định một tiến trình (từ mã nguồn)

Cơ chế	Cách triển khai hiện tại	Tác động khi có nhiều hơn một replica
Bus sự kiện thời gian thực	Một tập publish/subscribe trong bộ nhớ ở cấp module cấp dữ liệu cho luồng Server-Sent Events: cập nhật giám sát, tiến độ từng bước workflow và thông báo theo người dùng.	Sự kiện được publish trên một replica sẽ không đến được trình duyệt đang kết nối với replica khác.
Khoá đăng nhập & giới hạn đăng ký	Map trong bộ nhớ: 5 lần đăng nhập sai trong 10 phút sẽ khoá tài khoản 15 phút; đăng ký giới hạn 10 lần mỗi giờ cho mỗi địa chỉ client, theo header của proxy.	Bộ đếm sẽ tách riêng theo replica, nên giới hạn thực tế bị nhân lên.
Khôi phục workflow sau sự cố	Khi khởi động, mọi lượt chạy <code>running</code> được đánh dấu lại thành <code>resumable</code> . Điều này chỉ đúng khi tiến trình đang khởi động là bộ thực thi duy nhất.	Một replica thứ hai khởi động sẽ đánh dấu lại các lượt chạy vẫn đang chạy trên replica thứ nhất. Journal ghi (bên dưới) ngăn việc ghi ra ngoài bị trùng, nhưng các lượt chạy sẽ bị xáo trộn.
Bộ lập lịch	Không có cron trong tiến trình. Một tác vụ HĐH bên ngoài gửi POST đến endpoint tick mỗi phút.	Dùng được với nhiều replica, vì việc nhận lịch là nguyên tử trong SQL. Nguồn tick vẫn cần chuyển thành một job được quản lý.
Cache & throttle	Cache khám phá tool MCP (~30 giây cho mỗi người dùng) và throttle cho các dịch vụ địa lý công khai nằm trong bộ nhớ.	Trùng lặp không gây hại; hiệu quả giảm nhẹ.

4.2 Những gì đã sẵn sàng mở rộng

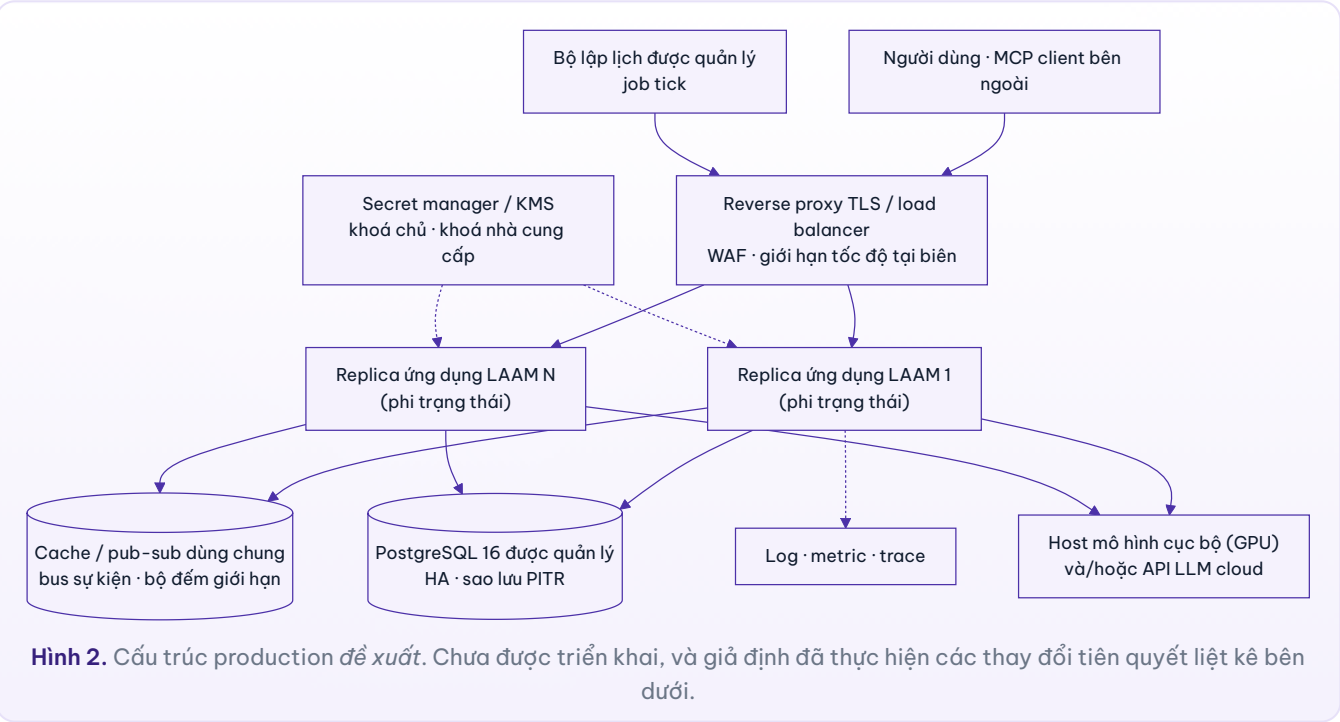
- **Trạng thái nằm trong PostgreSQL.** Phiên là JWT phi trạng thái. Hội thoại, lời gọi tool, lượt chạy và các bước workflow đều được lưu bền vững. Container ứng dụng không giữ trạng thái cục bộ nào ngoài các mục nêu trên.
- **Việc nhận lịch và tiếp tục chạy an toàn khi đồng thời.** Một transaction duy nhất vừa chèn lượt chạy vừa đẩy lịch tới mốc kế tiếp. Khoá duy nhất (`schedule`, `slot`) ngăn kích hoạt hai lần. Các lượt chạy

resumable được nhận bằng `FOR UPDATE SKIP LOCKED`.

- **Ghi ra ngoài có tính idempotent.** Mỗi thao tác ghi của workflow nhận trước một dòng trong bảng idempotency ghi trước (`UNIQUE(run, node, iteration)`) rồi mới gửi. Một thao tác ghi đã hoàn tất sẽ phát lại đầu ra đã lưu. Một thao tác ghi đã được nhận nhưng chưa bao giờ được ghi nhận kết quả sẽ báo lỗi rõ ràng và không bao giờ bị gửi lại.
- **Làm mới OAuth được tuần tự hoá giữa các tiến trình.** Một advisory lock của PostgreSQL bảo vệ việc xoay vòng refresh token cho các nhà cung cấp cấp refresh token dùng một lần.
- **Mức đồng thời của workflow có giới hạn.** Mỗi lượt chạy có ngân sách 200 bước và 100 phần tử foreach. Chế độ song song chạy tối đa 2 lời gọi mô hình và 6 lời gọi I/O đồng thời, fan-out tối đa 12.

4.3 Cấu trúc production mục tiêu PROPOSED

Cấu trúc này là một đề xuất. Chưa phần nào được triển khai. Nó giữ nguyên mã nguồn ứng dụng và chuyển các cơ chế một tiến trình sang hạ tầng dùng chung.



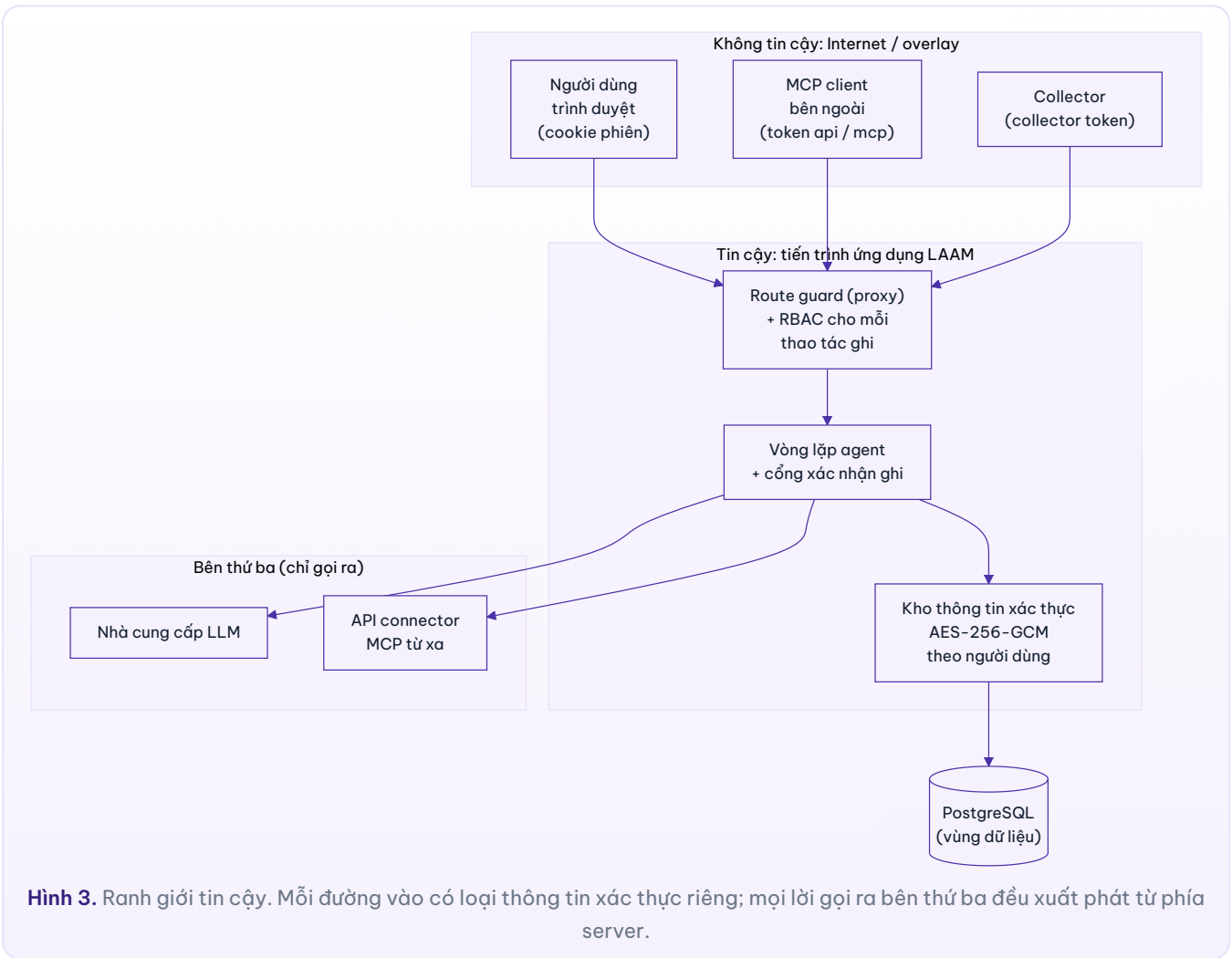
Thay đổi tiên quyết	Lý do
Chuyển bus sự kiện sang pub/sub dùng chung (vd. Redis hoặc PostgreSQL <code>LISTEN/NOTIFY</code>)	Để cập nhật SSE đến được người dùng trên bất kỳ replica nào.
Chuyển bộ giới hạn tốc độ (rate limiting) và khoá đăng nhập sang kho dùng chung, bổ sung giới hạn toàn API tại biên	Để giới hạn có hiệu lực trên mọi replica và bao phủ mọi endpoint.
Thay việc quét lượt chạy mở coil lúc khởi động bằng lease/heartbeat cho từng lượt chạy	Để một replica khởi động lại không thể đánh dấu lại các lượt chạy đang hoạt động của replica khác.
Thay tác vụ tick của HĐH bằng một job lập lịch được quản lý	Để loại bỏ sự phụ thuộc vào một host duy nhất.

Thay đổi tiên quyết	Lý do
PostgreSQL được quản lý với sao lưu tự động và khôi phục theo thời điểm; secret đặt trong secret manager hoặc KMS	Để đảm bảo độ bền dữ liệu, diễn tập khôi phục, xoay vòng khoá và vật liệu khoá riêng theo tenant.
Chạy migration như một job phát hành trong pipeline CI/CD	Để các lần deploy có thể lặp lại và audit được.

Số liệu năng lực (người dùng đồng thời, request mỗi giây) chưa được đo và không được tuyên bố. **TO BE COMPLETED** sau khi kiểm thử tải.

5 Kiến trúc bảo mật

LAAM là ứng dụng **cho một tổ chức duy nhất**, có phân quyền theo vai trò. Đây không phải SaaS đa tenant. Phần lớn thiết kế bảo mật xoay quanh một câu hỏi: làm sao để một AI agent có thể hành động thông qua các hệ thống mà người dùng đã kết nối không hành động khi chưa có sự phê duyệt rõ ràng của con người trong chat, và chỉ hành động trong giới hạn đã được duyệt trước (tool an toàn cho workflow, người nhận trong whitelist) khi workflow chạy không giám sát.



■ 5.1 Xác thực

- **Email và mật khẩu** qua Auth.js v5 Credentials. Mật khẩu được băm bằng **bcrypt**, dữ liệu đầu vào được kiểm tra bằng **zod**.
- **Phiên JWT**. Mỗi lần làm mới, token được **kiểm tra lại với cơ sở dữ liệu**. Người dùng bị vô hiệu hoá hoặc bị xoá sẽ bị đăng xuất ở request kế tiếp, và thay đổi vai trò có hiệu lực mà không cần đăng nhập lại.
- **Khoá tài khoản**: 5 lần sai trong 10 phút sẽ khoá tài khoản 15 phút. Tài khoản bị khoá, không tồn tại hoặc bị vô hiệu hoá đều trả về cùng phản hồi như sai mật khẩu.
- **Đăng ký** được điều khiển bởi `REGISTER_MODE` (open / invite / closed), kèm giới hạn 10 lần đăng ký mỗi giờ cho mỗi địa chỉ client (lấy từ header của proxy).
- **Route guard**: proxy của Next.js 16 (trước đây là middleware) chạy kiểm tra phân quyền an toàn cho edge trên mọi đường dẫn không tĩnh. Chỉ trang giới thiệu, trang đăng nhập và đăng ký, các endpoint của Auth.js và ba endpoint dành cho máy là công khai. Mỗi endpoint dành cho máy có cơ chế xác thực riêng: collector ingest, MCP server và tick của bộ lập lịch.
- Hiện chưa có SSO hay đăng nhập mạng xã hội (Mục 8); hỗ trợ OAuth của LAAM dành cho *connector* (Mục 5.5).

■ 5.2 Phân quyền

- Có bốn vai trò: **owner**, **admin**, **member**, **viewer**. Các route thay đổi trạng thái dùng chung hoặc nghiệp vụ (lướt chat, hội thoại, workflow, lướt chạy, lịch, connector, MCP server, agent tùy chỉnh) đều gọi `requireMutator`, nên viewer nhận HTTP 403. Viewer vẫn có thể sửa hồ sơ và thông báo của chính mình, thu hồi token của chính mình, và dùng các tiện ích phi trạng thái (phân tích tài liệu, TTS, soạn workflow bằng AI). Quản trị người dùng và máy yêu cầu owner hoặc admin.
- Các guard: không tự đổi vai trò của chính mình, không hạ cấp owner hoạt động cuối cùng, admin không được tạo khoá cho owner/admin. Thay đổi vai trò, vô hiệu hoá và cấp token đều được ghi vào `audit_log`.
- **Off-boarding** là một transaction duy nhất: đặt `disabled_at`, thu hồi mọi access token và thông tin xác thực collector của người dùng, và ghi một mục audit. Việc xác minh token cũng kiểm tra lại cờ vô hiệu hoá của chủ sở hữu, như một lớp phòng thủ bổ sung.

■ 5.3 Cô lập dữ liệu giữa người dùng

Dữ liệu riêng của mỗi người dùng được **giới hạn theo người dùng đó trong mọi truy vấn**: hội thoại, workspace, agent tùy chỉnh, workflow, lướt chạy và lịch, thông tin xác thực connector, cấu hình MCP server và thông báo. Dữ liệu giám sát lấy từ phiên của coding agent được **chia sẻ trong toàn tổ chức theo thiết kế** (mục đích của khu vực này chính là để cả nhóm cùng thấy); các phiên tạo qua API/MCP được lọc theo chủ thể trên luồng trực tiếp.

■ 5.4 Mã hoá thông tin xác thực của connector

- **Thuật toán:** AES-256-GCM với IV ngẫu nhiên 96 bit và thẻ xác thực, nên mọi hành vi sửa đổi đều làm giải mã thất bại.
- **Khoá theo người dùng:** HKDF-SHA256(master, salt, info = user id). Dữ liệu định dạng hiện hành (v2) mã hoá cho người dùng A **không thể giải mã** bằng khoá của người dùng B, nên một lỗi truy vấn hay lỗi mã khiến dữ liệu chéo người dùng sẽ thất bại theo hướng an toàn.
- **Phạm vi:** token connector, quyền OAuth đã cấp và cấu hình MCP server từ xa, bao gồm token xác thực và header của chúng. Dữ liệu lưu trữ mang dấu phiên bản. Dữ liệu định dạng cũ dùng một khoá toàn cục cho đến khi được mã hoá lại dần ở lần ghi kế tiếp.

Phạm vi thực tế

Mọi khoá theo người dùng đều được dẫn xuất từ một **secret chủ** nằm trong môi trường server: `CONNECTOR_KEY`, hoặc secret ký phiên `AUTH_SECRET` khi biến này không được đặt. Bắt buộc dùng một khoá chủ riêng là một hạng mục gia cố (Mục 8). Cơ chế này mang lại sự cô lập mật mã *giữa người dùng với nhau* dưới secret chủ đó. Nó **không** bảo vệ trước việc chính secret chủ bị lộ. Mã nguồn ghi nhận vật liệu khoá theo người dùng (KMS, hoặc bring-your-own-key) là việc tiếp theo, và định dạng dữ liệu có phiên bản là điểm nối để bổ sung.

Personal access token (loại `api`, `mcp` và `collector`) là giá trị ngẫu nhiên entropy cao. Mỗi token chỉ hiển thị một lần và chỉ được lưu dưới dạng hash SHA-256. Token hỗ trợ hết hạn và thu hồi, và được hiển thị bằng tiền tố cùng bốn ký tự cuối.

■ 5.5 OAuth cho connector

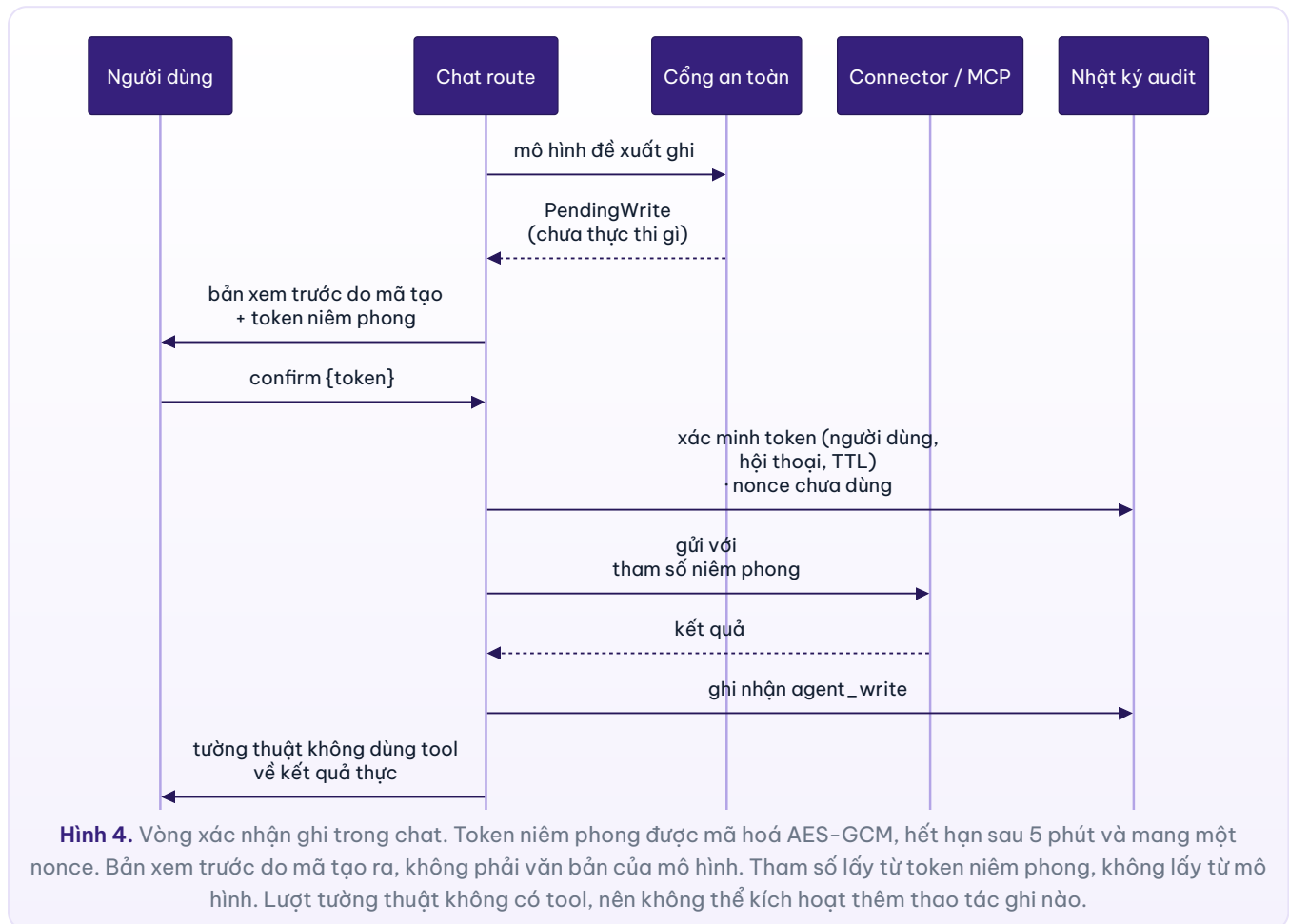
Google (Drive, Calendar, Gmail), Atlassian (Jira), Slack và Zalo kết nối qua luồng OAuth authorization code; Trello dùng luồng uỷ quyền riêng. Mỗi nhà cung cấp **bị tắt cho đến khi người vận hành cấu hình thông tin xác thực client của nó**. Cookie state riêng cho từng connector được đặt `httpOnly` và `SameSite=Lax`, thời hạn ngắn, để chống CSRF. PKCE được dùng khi nhà cung cấp hỗ trợ (Google, Zalo). Token được làm mới tại một điểm thực thi duy nhất, dưới advisory lock của cơ sở dữ liệu. Quyền bị thu hồi sẽ chuyển connector sang trạng thái "cần kết nối lại" thay vì thất bại âm thầm.

■ 5.6 An toàn request gọi ra ngoài (SSRF)

- **MCP server từ xa** là URL do người dùng cung cấp, được gọi phía server bằng token của người dùng. URL được kiểm tra hai lần. Khi lưu cấu hình, host được kiểm tra theo nguyên văn. Khi kết nối, **DNS được phân giải và mọi địa chỉ IPv4 và IPv6 thu được đều được kiểm tra**, loại bỏ các dải loopback, private, link-local, metadata cloud, IPv6 unique-local và IPv4-mapped.
- **Tool đọc URL** và cơ chế tải URL phía server chỉ nhận URL `http(s)` và đối chiếu hostname với một danh sách chặn gồm các tên loopback, private và link-local. Chúng tự xử lý chuyển hướng và kiểm tra lại ở mọi bước chuyển. Đưa mọi request gọi ra ngoài về một guard duy nhất có ghim kết quả DNS là một hạng mục gia cố (Mục 8).
- **Các connector danh mục** (danh mục public API và APILayer) cố định template URL của từng endpoint trong mã nguồn, nên mô hình chỉ có thể truy cập các URL do chính LAAM định nghĩa.
- **Giới hạn tài nguyên MCP:** tối đa 200 tool mỗi server, thời gian chờ mỗi lời gọi 15 giây, giới hạn cứng 200 nghìn ký tự mỗi kết quả kèm dấu hiệu cắt bớt hiển thị rõ, và giới hạn độ dài thông báo lỗi trả về cho mô hình.

5.7 Cổng xác nhận ghi cho hành động của AI

Đây là cơ chế an toàn trung tâm của LAAM. Trong chat, mô hình có thể đề xuất thao tác ghi nhưng không thể tự thực thi: mọi thao tác ghi vào hệ thống đã kết nối đều chờ người dùng xác nhận một bản xem trước do mã tạo, lời gọi đã xác nhận dùng tham số đã niêm phong, và nonce của nó được đối chiếu với nhật ký audit. Mọi tool được phân loại là đọc hoặc ghi, và tool nào có loại không xác định đều bị xem là ghi (fail-closed). Tool MCP từ xa chỉ được tính là đọc khi người dùng đã đánh dấu server là tin cậy và chính tool đó khai báo `readOnlyHint`.



- **Guard chống báo ghi sai:** nếu mô hình nói "đã gửi" hay "đã tạo" mà không có thao tác ghi nào thực sự được thực thi đằng sau, văn bản đó bị thay thế trước khi đến tay người dùng.
- **Workflow** (chạy theo lịch hoặc không giám sát) theo nguyên tắc fail-closed. Workflow chỉ được thực thi thao tác ghi với các tool được đánh dấu rõ là an toàn cho workflow. Tin nhắn gửi ra (e-mail, Slack, WhatsApp, Zalo) phải đến người nhận nằm trong whitelist theo từng kênh, và whitelist rỗng sẽ chặn việc gửi. Việc phân tích người nhận chặn tấn công chèn header e-mail. Node MCP trong workflow chỉ chạy các tool được người dùng tin cậy là đọc. Chạy thử (dry run) giả lập mọi thao tác ghi.
- **Soạn workflow bằng AI** thăm dò tool trong khi thao tác ghi bị chặn, không bao giờ thực thi.

5.8 Cấm thực thi mã

Agent không thể chạy mã. Một test trong bộ test quét mã nguồn của agent và workflow engine, và thất bại nếu việc tạo tiến trình, `eval`, `new Function` hay module `vm` xuất hiện. Test này chạy cùng bộ test; hiện chưa có CI (Mục 8). Tập tool nội bộ và năm loại node workflow được cố định trong mã, và cùng test đó kiểm tra tên và số lượng tool. Tool máy tính là bộ phân tích biểu thức tắt định, không dùng `eval`.

5.9 Header bảo mật và tăng truyền tải

Mọi phản hồi đều đặt `Strict-Transport-Security` (180 ngày, bao gồm subdomain), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, và một `Permissions-Policy` chặn camera, giới hạn micro và định vị chỉ trong origin của chính ứng dụng (cả hai cần cho giọng nói và bản đồ). **Content-Security-Policy chưa được đặt.** Việc này được hoãn lại chờ kiểm chứng trên trình duyệt (Mục 8). TLS được kết thúc tại mạng overlay.

5.10 Phơi bày MCP server

LAAM cũng là một MCP server: `POST /api/mcp`, Streamable HTTP. Server cung cấp **9 tool chỉ đọc** trên dữ liệu giám sát (phiên, máy, thống kê, nhật ký audit); không có thao tác ghi qua connector. Bên gọi cần personal access token loại `api` hoặc `mcp`; cookie phiên và collector token bị từ chối. Mỗi lời gọi được ghi nhận như một phiên được giám sát (best-effort). Hai hạng mục hiện **mới được dành chỗ, chưa được thực thi**: `scopes` của token được lưu nhưng chưa được kiểm tra, và ngoài tool audit vốn được lọc theo chủ thể, các tool đọc dữ liệu giám sát chia sẻ trong toàn tổ chức.

5.11 Endpoint dành cho máy

- Collector ingest:** bearer collector token gắn với một máy và giới hạn thân request 5 MB được áp dụng ngay khi đang stream. Collector bỏ đường dẫn file transcript của từng phiên trước khi tải lên; đường dẫn dự án và các đoạn văn bản ngắn vẫn được gửi. Các mẫu thông tin xác thực phổ biến được che khỏi đầu ra của sub-agent trước khi lưu.
- Tick của bộ lập lịch:** khi đã cấu hình secret, chỉ chấp nhận khớp secret header bằng phép so sánh thời gian hằng định. Khi không có secret, một kiểm tra request cục bộ chỉ dành cho môi trường phát triển được áp dụng; bắt buộc có secret khi cổng có thể truy cập từ máy khác (Mục 8, hạng mục 4).

5.12 Tổng hợp các biện pháp kiểm soát bảo mật

Biện pháp	Cách triển khai	Trạng thái
Xác thực mật khẩu	Hash bcrypt, đầu vào kiểm tra bằng zod, Auth.js v5 Credentials	LIVE
Quản lý phiên	JWT, kiểm tra lại với DB khi làm mới (vô hiệu hoá và thay đổi vai trò có hiệu lực ngay)	LIVE
Chống brute-force	Khoá đăng nhập (5 / 10 phút → 15 phút) và giới hạn đăng ký theo địa chỉ client do proxy báo; cả hai trong bộ nhớ	LIVE
Bảo vệ route	Kiểm tra tại edge proxy trên mọi đường dẫn không tĩnh; whitelist công khai tường minh	LIVE
RBAC	4 vai trò; <code>requireMutator</code> trên các route nghiệp vụ thay đổi trạng thái; guard cho owner/admin	LIVE
Cô lập dữ liệu theo người dùng	Truy vấn giới hạn theo chủ sở hữu cho chat, workflow, connector, agent, thông báo	LIVE
Off-boarding	Vô hiệu hoá trong transaction, thu hồi mọi token, ghi mục audit	LIVE
Mã hoá thông tin xác thực	AES-256-GCM, khoá theo người dùng bằng HKDF dưới một secret chủ; phản hồi được che	LIVE
Access token	Ngẫu nhiên, hash SHA-256, hiển thị một lần, có hết hạn và thu hồi	LIVE
OAuth cho connector	Cookie state (httpOnly, Lax), PKCE khi được hỗ trợ, làm mới có khoá	BUILT

Biện pháp	Cách triển khai	Trạng thái
Chống SSRF	Kiểm tra IP sau phân giải DNS cho MCP server khi kết nối; danh sách chặn hostname kèm kiểm tra chuyển hướng từng bước khi đọc URL	LIVE
Cổng xác nhận ghi của AI	Phân loại fail-closed; trong chat, xác nhận bản xem trước do mã tạo, tham số niêm phong, nonce đối chiếu với nhật ký audit	LIVE
An toàn ghi của workflow	Cờ an toàn cho workflow, whitelist người nhận, idempotency WAL, giả lập khi chạy thử	LIVE
Cấm thực thi mã	Test quét mã nguồn trong bộ test; tool nội bộ và loại node cố định trong mã	LIVE
Header bảo mật	HSTS, nosniff, chặn frame, referrer policy, permissions policy	LIVE
MCP server	Xác thực bằng token, tool chỉ đọc, lời gọi được audit như phiên	BUILT
Gia cố container	Người dùng non-root, runtime Alpine tối giản; SearXNG và TTS chỉ lắng nghe trên loopback	LIVE
Content-Security-Policy	Hoãn lại chờ kiểm chứng trên trình duyệt	PLANNED
Giới hạn tốc độ toàn API / phân tán	Chưa hiện thực (chỉ cho khu vực xác thực, trong bộ nhớ)	PLANNED
Thực thi scope của token	Scope được lưu, chưa được thực thi	PLANNED
SSO / định danh doanh nghiệp	Chưa hiện thực	PLANNED
Vật liệu khoá KMS / BYOK	Đã có điểm nối (dữ liệu có phiên bản); chưa hiện thực	PLANNED

Kiểm thử bảo mật: các biện pháp kiểm soát được bao phủ bởi unit test và integration test trong bộ Vitest của repository, gồm 3,573 test case (3,567 test đạt khi chạy lại trên phiên bản mã nguồn cơ sở, 2026-09-24). Chưa thực hiện kiểm thử xâm nhập độc lập. **TO BE COMPLETED**

6 Bảo vệ dữ liệu & luồng dữ liệu AI

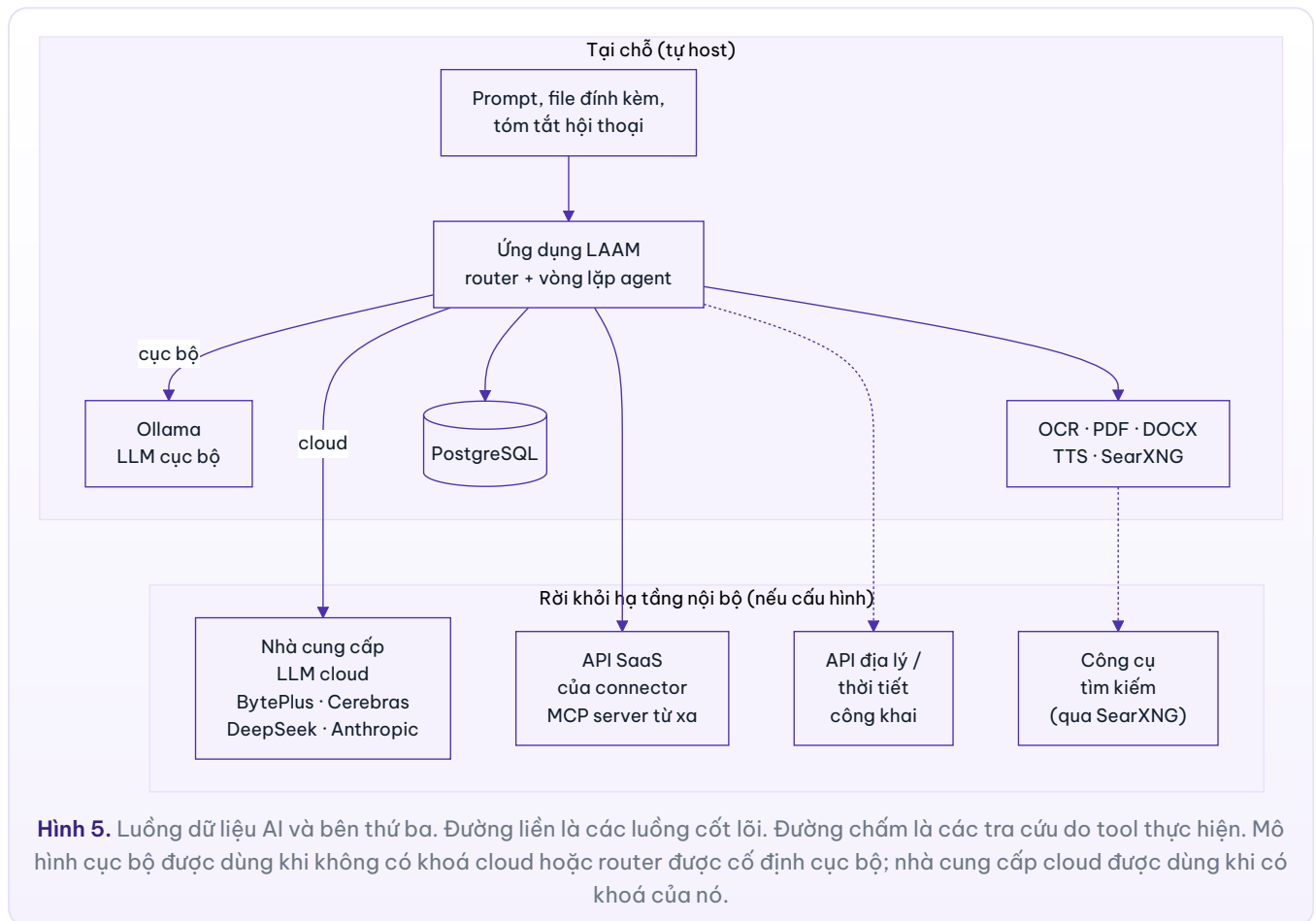
LAAM có thể chạy toàn bộ suy luận AI tại chỗ, nhưng đó là **một lựa chọn cấu hình, không phải mặc định**. Ngay khi khoá của một nhà cung cấp cloud được cấu hình, mọi tác vụ AI chạy nền chuyển sang nhà cung cấp đó, và mô hình chat mặc định cũng vậy khi nhà cung cấp là BytePlus hoặc Cerebras. Mục này liệt kê mọi đường mà dữ liệu có thể rời khỏi host.

6.1 Dữ liệu gửi đến mô hình ngôn ngữ

Mỗi lượt chat gửi cho mô hình system prompt: persona, ngày, ngôn ngữ, danh mục tool và các chỉ dẫn do MCP server của người dùng cung cấp. Lượt chat cũng gửi lịch sử hội thoại trong phạm vi ngân sách, bản tóm tắt cuốn chiếu các lượt cũ hơn, tin nhắn của người dùng và văn bản trích xuất từ file đính kèm. Khi dùng tính năng thị giác trên mô hình cục bộ, tối đa hai ảnh được gửi. **Kết quả tool** được trả lại cho mô hình trong cùng vòng lặp. Kết quả dạng bảng lớn được rút gọn thành bản tóm lược (digest) do mã tính toán (số lượng, tổng và các dòng mẫu) trước khi gửi, còn toàn bộ các dòng vẫn nằm trong PostgreSQL. Các mẫu thông tin xác thực phổ biến (khóa trong query string, bearer token, token GitHub) được che khỏi đầu ra của tool trước khi đến mô hình.

6.2 Quy tắc định tuyến mô hình

- **Chat tương tác:** người dùng chọn mô hình. Mô hình cloud chỉ xuất hiện khi đã đặt khoá; mặc định được chọn sẵn là BytePlus nếu đã cấu hình, sau đó đến Cerebras, nếu không thì dùng mô hình cục bộ. **Không có cơ chế tự động chuyển sang nhà cung cấp khác một cách âm thầm:** lỗi của nhà cung cấp được báo lại, không bao giờ bị gửi lại sang nơi khác.
- **Tác vụ AI chạy nền** (tóm tắt hội thoại, node agent trong workflow, soạn và rà soát workflow bằng AI): dùng `INTERNAL_MODEL` nếu được đặt; nếu không, dùng nhà cung cấp cloud đầu tiên được cấu hình (BytePlus → Cerebras → DeepSeek → Anthropic); chỉ dùng mô hình cục bộ khi không có khoá cloud nào.



Hình 5. Luồng dữ liệu AI và bên thứ ba. Đường liền là các luồng cốt lõi. Đường chấm là các tra cứu do tool thực hiện. Mô hình cục bộ được dùng khi không có khoá cloud hoặc router được cố định cục bộ; nhà cung cấp cloud được dùng khi có khoá của nó.

6.3 Dữ liệu khác rời khỏi host

- **Connector và MCP server từ xa:** tham số của mỗi lời gọi tool mà người dùng thực hiện hoặc phê duyệt được gửi đến endpoint SaaS hoặc MCP tương ứng.
- **Nhập liệu bằng giọng nói:** chuyển giọng nói thành văn bản dùng Web Speech API của trình duyệt, nên âm thanh được xử lý bởi dịch vụ của nhà cung cấp trình duyệt. Một nhà cung cấp Whisper tự host đang ở trạng thái **PLANNED**, đặt sau giao diện STT sẵn có. **Tổng hợp** giọng nói được tự host.
- **Truy vấn tìm kiếm web** đi đến các công cụ tìm kiếm thượng nguồn qua SearXNG tự host. **Tool đọc URL** tải các trang công khai do mô hình chọn.
- **Bản đồ và thời tiết:** các dịch vụ công khai không cần khoá về geocoding, chỉ đường, điểm quan tâm và thời tiết nhận tên địa điểm hoặc toạ độ trong request.
- **Phân tích tài liệu** (PDF, DOCX, OCR) chạy hoàn toàn bên trong container ứng dụng.

6.4 Dữ liệu lưu trữ

- Toàn bộ dữ liệu ứng dụng nằm trong PostgreSQL: hội thoại, lời gọi tool và kết quả được lưu, lượt chạy và đầu ra workflow, cùng metadata giám sát. Thông tin xác thực của connector và MCP được mã hoá ở tầng ứng dụng (Mục 5.4). Mã hoá volume cơ sở dữ liệu phụ thuộc vào host hoặc dịch vụ được quản lý. **TO BE COMPLETED** cho môi trường mục tiêu.
- Collector giám sát tải lên **metadata phiên** đã phân tích (số lượng, token, chi phí, thống kê tool, đường dẫn dự án) và các đoạn văn bản ngắn (hoạt động mới nhất, prompt của sub-agent tối đa 600 ký tự, và đầu ra của sub-agent tối đa 500 ký tự đã che các mẫu thông tin xác thực phổ biến). Collector không bao giờ tải lên toàn bộ transcript và bỏ đường dẫn file transcript.
- Chính sách lưu giữ dữ liệu chính thức và quy trình xuất hoặc xóa dữ liệu theo người dùng chưa được xác định. **TO BE COMPLETED**

Cấu hình hoàn toàn tại chỗ

Với dữ liệu không được phép rời khỏi hạ tầng nội bộ, người vận hành cấu hình: **(1)** không có khoá LLM cloud, hoặc `INTERNAL_MODEL` cố định vào một mô hình cục bộ và gỡ các mô hình cloud khỏi danh sách chọn; **(2)** chỉ dùng connector và MCP server tại chỗ; **(3)** tắt tìm kiếm web và các tool địa lý/thời tiết; và **(4)** tắt nhập liệu bằng giọng nói cho đến khi nhà cung cấp STT cục bộ được phát hành. Với các thiết lập này, suy luận chạy trên GPU cục bộ qua Ollama. Đánh đổi là chất lượng và tốc độ trả lời thấp hơn so với mô hình cloud.

7 Vận hành

7.1 Khả năng quan sát

Tín hiệu	Hiện đang có	Trạng thái
Log ứng dụng	Đầu ra <code>console</code> được thu thập thành log container (<code>docker logs</code>). Lỗi mang tiền tố gắn thẻ, ví dụ lỗi vòng lặp tool kèm id hội thoại. Chưa có logger có cấu trúc.	LIVE
Telemetry lượt chạy (trong sản phẩm)	Khu vực Giám sát là một read model trên chính các lượt chat và workflow của LAAM. Nó hiển thị mô hình thực sự được dùng, các tool được gọi, token và chi phí theo lượt chạy và theo bước, cùng tiến độ từng bước trực tiếp qua SSE.	LIVE
Audit lời gọi tool, cảnh báo	Mọi lời gọi tool trong chat đều được lưu; thao tác ghi đã phê duyệt đi vào <code>audit_log</code> ; cảnh báo trong ứng dụng cho phiên bị treo/tốn kém và kết quả workflow.	LIVE
Chất lượng AI	Một bộ đánh giá offline (kịch bản, bộ chấm điểm, test quy mô số lượng tool) ghi kết quả vào trang Reliability. Bộ này được chạy thủ công.	BUILT
APM / tracing / cảnh báo bên ngoài	Chưa hiện thực. <code>instrumentation.ts</code> chỉ dùng cho khôi phục sự cố lúc khởi động, không dùng cho telemetry.	PLANNED

7.2 Theo dõi chi phí

Lượng token sử dụng được ghi nhận theo từng tin nhắn của trợ lý và từng bước workflow, cùng với mô hình đã trả lời. Chi phí bằng USD được tính từ bảng giá theo mô hình được duy trì, bao gồm các mô hình của Anthropic, BytePlus, Cerebras và DeepSeek. **Mô hình chưa có giá sẽ trả về "unknown" thay vì một con số ước tính**, và không hiển thị chi phí khi nhà cung cấp không báo cáo lượng sử dụng. Chi phí xuất

hiện trên dashboard (theo mô hình, heatmap, KPI), trên từng lượt chạy workflow, và trong cảnh báo chủ động khi chi tiêu vượt ngưỡng. Mô hình cục bộ có chi phí \$0; bảng giá được duy trì thủ công.

7.3 Sao lưu và khôi phục

Hạng mục	Trạng thái
Script sao lưu	Đã có script <code>pg_dump</code> với cơ chế dọn dẹp giữ 14 ngày, kèm công thức lập lịch hằng ngày và quy trình khôi phục được ghi chép. Script vẫn trở đến container cơ sở dữ liệu dùng trước khi chuyển sang instance PostgreSQL dùng chung, nên phải trở lại trước khi có thể dựa vào. TO BE COMPLETED
Sao lưu instance dùng chung, diễn tập khôi phục, RPO/RTO	Sao lưu ở cấp instance được quản lý bên ngoài repository của LAAM và chưa được kiểm chứng tại đây; diễn tập khôi phục và mục tiêu RPO/RTO chưa được xác định. TO BE COMPLETED
Khôi phục workflow sau sự cố	Có sẵn. Lượt chạy bị gián đoạn do khởi động lại sẽ tiếp tục từ journal các bước, và thao tác ghi ra ngoài được bảo vệ bởi bảng idempotency (Mục 4.2). LIVE

7.4 Cập nhật và migration

- 1. Kéo bản phát hành và build image mới tại chỗ.
- 2. Áp dụng các migration còn chờ từ host (`drizzle-kit migrate`). Migration có journal và an toàn khi chạy lại.
- 3. Thay container ứng dụng (khởi động lại ngắn; deploy không gián đoạn cần các thay đổi ở Mục 4.3). Cơ sở dữ liệu không bị động đến.
- 4. Chạy checklist sau deploy: trang công khai, endpoint xác thực, đăng nhập thật trên trình duyệt, mô hình cục bộ, OCR, secret tick, và test của từng connector.

7.5 Health check

- **Health check của container:** cứ 30 giây image tải trang đăng nhập công khai một lần. Đây là probe liveness. Nó không kiểm tra kết nối cơ sở dữ liệu hay mô hình.
- **Tình trạng các phụ thuộc** được kiểm tra trong sản phẩm (khả dụng của OCR, thao tác "Test" của connector và MCP); tìm kiếm web và đo tài nguyên host suy giảm êm với trạng thái "không khả dụng" rõ ràng. Một endpoint readiness bao phủ cơ sở dữ liệu, mô hình và các dịch vụ vệ tinh đang ở trạng thái PLANNED.

8 Hạng mục gia cố trước khi lên production

Công việc đã lên kế hoạch cho việc ra mắt production, tổng hợp từ mã nguồn, chú thích trong mã, changelog và một đợt rà soát mã nội bộ. Các biện pháp kiểm soát bảo mật cốt lõi đã có trong mã nguồn; một số cần được siết chặt thêm (hạng mục 6 đến 10), và phần lớn công việc còn lại thuộc về **vận hành**.

#	Hạng mục	Thay đổi dự kiến	Lĩnh vực
1	Content-Security-Policy	Định nghĩa và áp dụng CSP sau khi kiểm chứng trên trình duyệt.	Web
2	Giới hạn tốc độ toàn API	Giới hạn theo người dùng / theo token trên mọi route; giới hạn tại biên ở proxy.	Lạm dụng
3	Trạng thái dùng chung cho replica	Bộ giới hạn dùng kho chung và pub/sub; lease cho từng lượt chạy.	Mở rộng
4	Bộ lập lịch được quản lý	Job tick được quản lý thay cho tác vụ HĐH; bắt buộc có secret tick.	Vận hành
5	Định danh	SSO OIDC/SAML, MFA tùy chọn; thực thi scope của token và lọc dữ liệu MCP theo người dùng; bản Auth.js ổn định.	Định danh
6	Quản lý khoá	Khoá chủ riêng lấy từ KMS / secret manager, bắt buộc khi khởi động, không có giá trị thay thế và tách biệt với secret ký phiên; có xoay vòng; mã hoá lại phần dữ liệu định dạng cũ còn lại; BYOK qua điểm nối dữ liệu có phiên bản.	Mật mã
7	Mặc định an toàn	Đăng ký chỉ qua lời mời; ứng dụng, giao diện quản trị và agent đo tài nguyên chỉ gắn với giao diện mạng riêng; chuyển chuỗi kết nối cơ sở dữ liệu từ file Compose vào <code>.env</code> .	Mạng
8	Guard cho request gọi ra ngoài	Một guard duy nhất có ghim kết quả DNS cho mọi request gọi ra ngoài (đọc URL, tải URL, MCP), bao phủ dải carrier-grade NAT và kiểm tra lại ở mọi lần chuyển hướng.	Mạng
9	Gia cố cổng xác nhận ghi	Nhận nonce theo cách nguyên tử (atomic) và thử lại idempotent cho thao tác ghi MCP đã xác nhận.	An toàn AI
10	Chi tiết xác thực	Khoá giới hạn tốc độ lấy từ địa chỉ client do proxy tin cậy cung cấp; thời gian phản hồi đăng nhập đồng nhất.	Định danh
11	Khả năng quan sát	Log có cấu trúc, tracing, cảnh báo, endpoint readiness.	Vận hành
12	Sao lưu và CI/CD	Trở lại sao lưu, diễn tập khôi phục, RPO/RTO; CI/CD chạy bộ test (gồm test guard cấm thực thi mã), quét (scanning) và migration.	Vận hành
13	Quyền riêng tư & đảm bảo	STT tự host; kiểm thử xâm nhập bởi bên thứ ba và rà soát cổng ghi của AI.	Đảm bảo